Lab 1 — Writing a PRD and a TRD with Prompt Engineering

After Module 1 — Prompt Engineering for Developers

You will not write any application code in this lab. You will produce two specification documents that Lab 2 builds from.

Step 1 — Create the folder

Create a folder named TodoApp on your machine. Inside it, create a subfolder named docs. Open the TodoApp folder in VS Code.

Step 2 — Open Copilot Chat in Ask mode

Press Ctrl + Alt + I to open Copilot Chat. From the mode picker at the bottom of the chat box, select Ask. From the model picker, select Claude Sonnet 5.

Ask mode is used because this lab produces documents through conversation, not file edits.

Step 3 — Generate the PRD

Type the following prompt into Copilot Chat and press Enter.

```
Role:     Senior product manager with 10 years of experience writing
          specifications for small web applications.
Task:     Write a Product Requirements Document for a Todo application.
Audience:  A developer who will implement this application, and a trainer
          who will review it.

Context:   This is a single-user application. There is no login and no
          multi-user support. It will be built in three versions:
           Version 1 - No database. Todos are held in server memory
                and are lost when the server restarts.
                Fields: Id, Title, Description, DateCreated,
                IsCompleted.
           Version 2 - Same functionality, but todos are stored in a
                SQLite database so they survive a restart.
           Version 3 - A Category field is added to each todo.

Constraints:
 - Describe WHAT the product does, never HOW it is built.
 - Do not mention any framework, library, or programming language.
 - Keep it under 800 words.
 - Version 1 is the only version being built now. Versions 2 and 3
   must appear in a Roadmap section as future scope.

Output format:
 1. Overview
 2. Target User
 3. In Scope for Version 1
```

4. Out of Scope for Version 1
5. Functional Requirements (numbered FR-1, FR-2, ...)
6. Non-Functional Requirements
7. Roadmap (Version 1, Version 2, Version 3)
8. Assumptions

Step 4 — Save the PRD

Create a file docs/PRD.md. Copy the PRD from the chat into it and save.

Step 5 — Generate the TRD

The TRD describes how the application will be built. The architecture is given to you below — do not ask Copilot to choose it.

```
Role:    Senior software architect.
Task:    Write a Technical Requirements Document for Version 1 of the
         Todo application described in the PRD below.

Architecture (already decided - do not propose alternatives):
 - Frontend is React. Backend is Python with FastAPI.
 - Single-user application. No authentication.
 - A single repository containing a frontend folder and a backend
   folder.
 - Three backend layers - routers, services, and a repository layer.
   The router handles HTTP only. The service holds business rules.
   The repository is the only code that touches data storage.
 - Version 1 holds todos in a Python list inside the repository layer.
   Version 2 replaces that list with SQLite via SQLAlchemy and Alembic.
   Only the repository layer changes when that happens.

Constraints:
 - Version 1 only. No database. No Category field. No authentication.
 - Fields on a todo: Id, Title, Description, DateCreated, IsCompleted.
 - REST API. JSON request and response bodies.
 - Python 3.12, FastAPI, Pydantic models.
 - React with functional components and useState.
 - Every requirement must trace back to a functional requirement
   number from the PRD.

Output format:
 1. Architecture Overview
 2. Folder Structure (as a directory tree)
 3. Data Model (field name, type, required, description)
 4. API Endpoints (method, path, request body, response body,
    status codes)
 5. Backend Layers (responsibility of each layer, and what it may
    not do)
 6. Frontend Components (name, responsibility, state it owns)
 7. Version 2 Migration Note - exactly which files change when SQLite
    is introduced
```

```
8. Traceability Table (PRD requirement number to TRD section)

Do not ask clarifying questions. Produce the document.

PRD follows:
[paste the contents of docs/PRD.md here]
```

Step 6 — Verify and save the TRD

Do not accept the first output. Send this follow-up.

```
Verify your own TRD.

1. For every functional requirement in the PRD, confirm there is an API
   endpoint and a frontend component that satisfies it. List any
   requirement with no matching implementation.
2. Confirm that no file outside the repository layer would need to
   change when SQLite is introduced in Version 2. If any would, name it
   and explain why.
3. List anything a developer could implement in two different ways
   because the document is not specific enough.

Then produce the corrected TRD.
```

Create a file docs/TRD.md. Copy the corrected TRD into it and save.

What you should have at the end of this lab

```
TodoApp/
  docs/
    PRD.md
    TRD.md
```

No application code. Two documents that a developer could hand to any team and get back the same application.